You

Can you summarise this for my lecture notes?

For the most part, generative models are developed in three phases:
Training, to create a foundation model that can serve as the basis of multiple gen AI applications.
Tuning, to tailor the foundation model to a specific gen AI application.
Generation, evaluation and retuning, to assess the gen AI application's output and continually improve its quality and accuracy.

Generative Models: Methods and applications
Generative AI is based on Deep Learning models, mainly:
Variational autoencoders (VAEs), which drove breakthroughs in image recognition, natural language processing and anomaly detection.
Generative adversarial networks (GANs) and diffusion models, which improved the accuracy of previous applications and enabled some of the first AI solutions for photo-realistic image generation.

Copilot

Absolutely! Here's a summary for your lecture notes:

Generative Models: Methods and Applications
Generative models are developed in three main phases:

1. **Training:** Establishing a foundation model for multiple generative AI applications.
2. **Tuning:** Customizing the foundation model for specific applications.
3. **Generation, Evaluation, and Retuning:** Continuously assessing and enhancing the application's output for better quality and accuracy.

Generative AI is based on Deep Learning models, including:

- **Variational Autoencoders (VAEs):** Enhanced image recognition, natural language processing, and anomaly detection.
- **Generative Adversarial Networks (GANs) and Diffusion Models:** Improved previous applications' accuracy and enabled AI solutions for photo-realistic image generation.

Does that capture everything you need for your lecture notes?

You

I'm trying to run an update sql command on W3schools, however I keep getting the following message, my browser is Chrome, how cabn I resolve it?
Message:

Error in SQL:
Operation must use an updateable query.
Note: Your browser does not support WebSQL, so you are now using a light-version of our Try-SQL Editor, with a read-only Database.

If you use a browser with WebSQL support (Chrome, Safari, or Opera), you can try any SQL statement, and play with the Database as much as you like.

Copilot

It sounds like you're encountering an issue with running an update SQL command on W3Schools YouTube. The error message you're seeing, "Operation must use an updateable query," typically occurs when the query you're trying to run is read-only access-programmers.co.uk.

Here are a few steps you can try to resolve this issue:

1. **Check Permissions**: Ensure that the database file is not set to read-only software-solutions-online.com. You can right-click on the file, select "Properties," and uncheck the "Read-only" attribute if it's checked software-solutions-online.com.
2. **Primary Key**: Make sure the table you're trying to update has a primary key software-solutions-online.com. Without a primary key, the update operation might not be allowed software-solutions-online.com.
3. **WebSQL Support**: Ensure that your browser supports WebSQL YouTube. Chrome, Safari, and Opera should support it, but you might want to double-check your browser settings or try updating Chrome to the latest version YouTube.
4. **Database Location**: Ensure that the database file is not located in a restricted folder like "Program Files" or "App_data" software-solutions-online.com. Move it to a location like "My Documents" if necessary software-solutions-online.com.
5. **Open Connections**: Make sure no other database connections are open that might be locking the table you're trying to update access-programmers.co.uk.

If you've tried these steps and are still encountering the issue, it might be helpful to check W3Schools' support or forums for additional guidance.

Does this help resolve your issue?

References

1. Fix An Error Occurred While Checking for Update Error in Google Chrome
2. Solved "Operation must use an updateable query" but only sometimes
3. What is “Operation Must Use an Updateable Query” and How Do You Fix It?

You

I can't find the database, I'm using the latest version of Google chrome?

Copilot

It sounds like you're having trouble locating the database file on W3Schools. Since you're using the latest version of Google Chrome, which supports WebSQL, the issue might be with how the database is accessed or configured.

Here are a few steps you can try:

1. **Check the URL**: Ensure you're accessing the correct URL for the W3Schools Try-SQL Editor w3schools.com. Sometimes, navigating to the wrong page can cause issues.
2. **Browser Settings**: Double-check your browser settings to ensure that JavaScript and cookies are enabled, as these are required for the Try-SQL Editor to function properly.
3. **Clear Cache**: Try clearing your browser cache and cookies, then reload the page. This can sometimes resolve issues with web applications.
4. **Try Another Browser**: Although Chrome supports WebSQL, you could try using another browser like Firefox or Edge to see if the issue persists.
5. **Check W3Schools Support**: Visit the W3Schools support or forums for any additional guidance or known issues with the Try-SQL Editor.

If none of these steps work, it might be helpful to reach out to W3Schools support directly for further assistance.

Does this help clarify things a bit?

References

1. SQL UPDATE Statement - W3Schools

You

The latest version of Chrome doesn't have webSQL support, so doesn't Firefox or Edge, what other browser can I use?

Copilot

You're right developer.chrome.com! The latest versions of Chrome, Firefox, and Edge no longer support WebSQL developer.chrome.com.

Instead, you can use **Opera** or **Safari** for WebSQL support yathit.github.io. Both of these browsers still support WebSQL, so you should be able to run your SQL commands without any issues.

Would you like more information on how to switch to one of these browsers?

References

1. Deprecating and removing Web SQL | Blog - Chrome Developers
2. Supported browsers and their limitation - Yathit

You

I got opera, but it still says it in Opera as well

Copilot

I'm sorry to hear that you're still having trouble! Since WebSQL support is limited to older versions of browsers, it might be worth considering alternative methods for running your SQL commands.

You could try using **SQLite compiled to WebAssembly** developer.chrome.com. This approach allows you to run SQLite directly in the browser without relying on WebSQL developer.chrome.com. There are tools like **DuckDB-Wasm** and **absurd-sql** that can help with this developer.chrome.com.

Would you like more information on how to set this up?

References

1. Deprecating and removing Web SQL | Blog - Chrome Developers

You

What chrome browser extensions are available to achieve this?

Copilot

Since WebSQL support has been deprecated in most modern browsers, including Chrome, you might want to consider using browser extensions that allow you to run SQLite queries directly in your browser. Here are a few options:

1. **SQLite Manager**: This extension provides a user-friendly interface to manage SQLite databases directly in your browser.
2. **DuckDB-Wasm**: This tool allows you to run DuckDB, an in-memory analytical database, using WebAssembly, which can be useful for running SQL queries.
3. **absurd-sql**: Another WebAssembly-based tool that provides SQL capabilities in the browser.

Would you like more details on how to install and use any of these extensions?

You

How can I create a webpage that works as a template for a flask method in my python script that takes a multiline text input from a textbox in the top left of the webpage (under the main heading), another multiline text input from a textbox in the top right of the webpage, send these 2 as 'template' and 'data' to the python method, also include a full width textbox for the output from the python and then a copy button at the bottom?

Copilot

Sure, I can help you create a webpage template using Flask that takes two multiline text inputs and sends them to a Python method. Here's a simple example:

First, create your Flask app in a file named app.py:

```
from flask import Flask, render_template, request, jsonify

app = Flask(__name__)

@app.route('/')
def home():
    return render_template('index.html')

@app.route('/process', methods=['POST'])
def process():
    template = request.form['template']
    data = request.form['data']
    # Process the inputs as needed
    output = f"Processed Template: {template} with Data: {data}"
    return jsonify({'output': output})

if __name__ == '__main__':
    app.run(debug=True)
```

Then, create the HTML template in a file named templates/index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Flask Template</title>
    <style>
        body {
            font-family: Arial, sans-serif;
        }
        .container {
            display: flex;
            flex-direction: column;
            align-items: center;
            margin-top: 50px;
        }
        .inputs {
            display: flex;
            justify-content: space-between;
            width: 100%;
        }
        textarea {
            width: 45%;
            height: 150px;
        }
        .output {
            width: 100%;
            margin-top: 20px;
        }
    </style>
    <script>
        function copyToClipboard() {
            const outputText = document.getElementById("output-text").value;
            navigator.clipboard.writeText(outputText).then(() => {
                alert("Copied to clipboard");
            });
        }

        async function submitForm(event) {
            event.preventDefault();
            const formData = new FormData(event.target);
            const response = await fetch('/process', {
                method: 'POST',
```

```
            body: formData
        });
        const result = await response.json();
        document.getElementById("output-text").value = result.output;
    }
  </script>
</head>
<body>
  <div class="container">
    <h1>Main Heading</h1>
    <form onsubmit="submitForm(event)">
      <div class="inputs">
        <textarea name="template" placeholder="Template Input"></textarea>
        <textarea name="data" placeholder="Data Input"></textarea>
      </div>
      <div class="output">
        <textarea id="output-text" readonly placeholder="Output will be displayed here"></textarea>
      </div>
      <button type="submit">Submit</button>
      <button type="button" onclick="copyToClipboard()">Copy</button>
    </form>
  </div>
</body>
</html>
```

This example sets up a simple Flask application that:

1. **Renders a webpage with two multiline text input boxes** (one on the left and one on the right).
2. **Sends the input values to the Flask method** when the form is submitted.
3. **Processes the inputs** and returns the result.
4. **Displays the result in a full-width output box** below the inputs.
5. **Provides a copy button** to copy the output text to the clipboard.

Just save these files and run your Flask app with python app.py. Open your browser to http://127.0.0.1:5000/ to see the webpage and test the functionality. github.com github.com

References

1. github.com
2. github.com

You

(Now Wednesday, Nov 20)
How can I write a follow-up to this email mentioning that I've made decent progress and the demo should be ready in 1 or 2 months then we can progress from there hopefully:
Hi all,

I hope this message finds you well.

I am pleased to inform you that I have resumed my work at the Trust. You may recall that before my departure, I was developing an AI document extraction program. I have now returned to this project, and I anticipate that it should be ready for demonstration within the next two to three months.

I will provide an update on my progress in approximately one month.

Thank you for your continued support.

Best regards, Hasan Akhtar

Copilot

Certainly, here is a follow-up to your email with the requested information:

Hi all,

I hope this message finds you well.

I am pleased to share that I have made significant progress on the AI document extraction program. I anticipate that the demo will be ready within the next one to two months, after which we can move forward from there, hopefully.

I will provide another update on my progress in approximately one month.

Thank you for your continued support.

Best regards, Hasan Akhtar

Does this align with what you had in mind?

You

how can I adapt it for a Teams message similar to this previous one:

Hi Katharine,

Just letting you know I'm back at the Trust and have resumed work on the AI document extraction program. I expect it will be ready for demonstration in the next two to three months. I'll provide an update on my progress in about a month.

Thanks for your continued support!

Best regards, Hasan

Copilot

Sure, here's an adapted version for your Teams message:

Hi Katharine,

Just a quick update to let you know I've made decent progress on the AI document extraction program. I expect the demo should be ready in the next one to two months, and we can hopefully move forward from there. I'll share another update on my progress in about a month.

Thanks for your continued support!

Best regards, Hasan

Does this suit your needs?

You

I'm trying to use gemini in python, I currently have this code but which other models are available to use?

```
credentials, project_id = google.auth.default()

vertexai.init(project="generalpurposeai", location="us-central1")

model = GenerativeModel(model_name="gemini-1.0-pro")
```

Copilot

Great to hear you're working with Gemini in Python! There are several models available for use with the Gemini API ai.google.dev. Here are some of the options:

1. **Gemini 1.5 Flash**: A balanced multimodal model that handles audio, images, video, and text inputs ai.google.dev. It's optimized for a variety of tasks ai.google.dev.
2. **Gemini 1.5 Flash-8B**: A faster and more cost-efficient version of the Flash model, ideal for high-frequency tasks ai.google.dev.
3. **Gemini 1.5 Pro**: The highest performing model for complex reasoning tasks requiring more intelligence ai.google.dev.
4. **Gemini 1.0 Pro**: A natural language model for multi-turn text and code chat, and code generation ai.google.dev.
5. **Text Embedding**: Used for measuring the relatedness of text strings ai.google.dev.
6. **AQA**: Provides source-grounded answers to questions ai.google.dev.

You can specify the model you want to use in your code like this:

```
model = GenerativeModel(model_name="gemini-1.5-flash")
# or
model = GenerativeModel(model_name="gemini-1.5-pro")
# or any other model variant
```

Would you like more details on any specific model or how to use them?

References

Gemini models | Gemini API | Google AI for Developers

You

Which would the best model to use for writing cover letters based on a user's full CV/Resume and the full job description?

Copilot

For writing cover letters based on a user's full CV/Resume and the job description, **GPT-4** would be an excellent choice hyperwriteai.com. It's known for its ability to generate coherent and contextually relevant text, making it ideal for crafting personalized cover letters.

Would you like some help integrating GPT-4 into your Python script?

References

Cover Letter Generator | AI-powered cover letter creator - HyperWrite | AI

You

Out of those gemini models, I'm saying?

Copilot

Got it! Among the Gemini models, **Gemini 1.0 Pro** would be the best choice for writing cover letters based on a user's CV/Resume and the job description. This model is designed for multi-turn text and code chat, and it excels at generating coherent, contextually relevant text. It's well-suited for tasks that require detailed language generation and personalization.

Here's how you can set up your code to use **Gemini 1.0 Pro** for this purpose:

```
from google.auth import default
import vertexai
from vertexai.language_models import GenerativeModel

credentials, project_id = default()
```

```
vertexai.init(project="generalpurposeai", location="us-central1")

model = GenerativeModel(model_name="gemini-1.0-pro")

# Define your function to generate the cover letter
def generate_cover_letter(cv_text, job_description):
    prompt = f"Write a cover letter based on the following CV and job description.\n\nCV:\n{cv_text}\n\nJob
Description:\n{job_description}"
    response = model.generate(text=prompt)
    return response['text']

# Example usage
cv_text = "Your CV text here"
job_description = "The job description here"
cover_letter = generate_cover_letter(cv_text, job_description)
print(cover_letter)
```

This script will generate a personalized cover letter based on the provided CV and job description.

Does this help with your project?

You

This is my current prompt, how could it be improved to receive a response that is natural-sounding and flows like a manually written cover letter?

```
prompt = f"""Given the contents of my CV (Resume) and the job description which are below:
        \nMy Resume/CV is as follows: {resume}
        The job description is as follows: {job_description}
        \nPlease generate an appropriate cover letter."""
```

Copilot

To make your prompt more effective at generating a natural-sounding and cohesive cover letter, you can include additional context and instructions. Here's an improved version of your prompt:

```
prompt = f"""You are an experienced and professional writer. Using the information provided in my CV (Resume) and
the job description below, please write a natural-sounding and engaging cover letter. The cover letter should highlight
my relevant skills and experiences, match the tone of a professional application, and demonstrate my enthusiasm for the
position.

My CV/Resume:
{resume}

Job Description:
{job_description}

Cover Letter:
"""
```

This revised prompt:

1. **Sets the context** by specifying that the response should come from the perspective of an experienced and professional writer.
2. **Clarifies the desired output** by asking for a natural-sounding and engaging cover letter.
3. **Provides guidance** on the tone and content of the cover letter, ensuring it highlights relevant skills and experiences, aligns with a professional application, and shows enthusiasm for the position.

These adjustments should help you receive a more polished and effective cover letter that feels like it was manually written.

You

How can I specify the Gemini 1.5 Flash-8B model in my python code?

Copilot

You can specify the **Gemini 1.5 Flash-8B** model in your Python code by setting the model_name parameter to "gemini-1.5-flash-8b" when initializing the GenerativeModel. Here's an example of how you can do this:

```
from google.auth import default
import vertexai
from vertexai.language_models import GenerativeModel

credentials, project_id = default()

vertexai.init(project="generalpurposeai", location="us-central1")

# Specify the Gemini 1.5 Flash-8B model
model = GenerativeModel(model_name="gemini-1.5-flash-8b")

# Define your function to generate the cover letter
def generate_cover_letter(cv_text, job_description):
    prompt = f"""You are an experienced and professional writer. Using the information provided in my CV (Resume) and the job description below, please write a natural-sounding and engaging cover letter. The cover letter should highlight my relevant skills and experiences, match the tone of a professional application, and demonstrate my enthusiasm for the position.

    My CV/Resume:
    {cv_text}

    Job Description:
    {job_description}

    Cover Letter:
    """
    response = model.generate(text=prompt)
    return response['text']

# Example usage
cv_text = "Your CV text here"
job_description = "The job description here"
cover_letter = generate_cover_letter(cv_text, job_description)
print(cover_letter)
```

This code will use the **Gemini 1.5 Flash-8B** model to generate a cover letter based on the given CV/Resume and job description. Let me know if you need any further assistance!

You

(Now Friday, May 2)
How can I write a description for this code that can be used to aid a brief that I'm writing where I make a better version of the app with a flask front-end that is user friendly and generates naturally flowing cover letters for users to send to potential employers when applying for jobs?

Copilot

Here's a description that you can use to support your brief:

Project Overview: AI-Powered Cover Letter Generator with Flask Front-End

This project aims to enhance the functionality and user experience of an AI-driven cover letter generator by integrating a Flask-based front-end. The improved application will allow users to easily input their CV/Resume and the job description, generating professional, naturally flowing cover letters tailored for specific job applications.

Current Implementation:

The existing implementation utilizes **Gemini AI models** in Python to generate cover letters based on a structured prompt. Users provide their CV and the job description, and the AI model processes this data to create a draft cover letter. However, the current setup lacks an intuitive user interface and requires manual input through a script.

Proposed Enhancements:

1. **User-Friendly Flask Front-End**
    - A responsive web interface with text fields for CV input, job description, and output display.
    - A clean design with clear instructions, ensuring ease of use for all users.
2. **Improved Cover Letter Generation**
    - Optimized AI prompts to ensure the generated text flows naturally and reads professionally.
    - Enhanced formatting, ensuring the structure and tone align with standard cover letter conventions.
3. **Interactivity & Additional Features**
    - Real-time editing and refinement options before sending the letter.
    - A **copy-to-clipboard** button for easy access.
    - Potential email integration for direct application submission.

Expected Impact:

This revamped tool will **simplify the job application process**, providing users with a well-crafted, **AI-assisted cover letter** that enhances their chances of securing interviews. By combining **Flask and Gemini AI**, we aim to create a **seamless and efficient** solution that automates the most time-consuming part of applying for jobs: writing a compelling cover letter.

Would you like any additional details or refinements for your brief?

You

Can you give it to me as bulleted notes briefly explaining what the old version was about, how it was implemented and how the new version should be implemented with improvements?

Copilot

Sure! Here's a concise breakdown:

Old Version: AI Cover Letter Generator

- **Purpose:** Generate a cover letter based on a user's CV and job description using AI.
- **Implementation:**
    - Python script utilizing **Gemini AI models**.
    - Takes CV and job description as text inputs.
    - Generates cover letter based on predefined prompts.
- **Limitations:**
    - **No user-friendly interface** (requires manual input via script).
    - **Limited customization** (users can't edit easily before finalizing).
    - **No real-time feedback or interaction** with AI.

New Version: Flask-Based AI Cover Letter Generator

- **Key Improvements:**
    - **Web-Based Interface:** A Flask front-end for easy user interaction.
    - **Better Cover Letter Quality:** Optimized prompts to ensure natural, professional tone.
    - **Enhanced Customization:** Users can edit and refine the AI-generated cover letter before finalizing.
- **Implementation Plan:**
    - **Flask Web App**
        - Create a clean, user-friendly UI with input fields for CV and job description.
    - **AI Integration**
        - Use **Gemini 1.5 Flash-8B** for high-quality text generation.
        - Craft prompts ensuring flow, structure, and relevance.
    - **Interactivity**

        - Live preview of AI-generated cover letter.
        - Options for users to edit before finalizing.
    - **Additional Features**
        - **Copy-to-clipboard** functionality for quick access.
        - **Download as PDF** option.
        - **Potential email integration** for direct job application submission.

Expected Outcome

- A **fully automated yet customizable** tool that makes job applications easier.
- **Professional, well-structured cover letters** that feel like they were written manually.
- **Efficient and intuitive** user experience with minimal effort required.

Would you like me to refine any specific details?

You

How can I do a brief for the May project (AI Cover Letter Writer) based on these notes from the previous version and new implementation methods?

Copilot

Here's a structured brief for the **May Project: AI Cover Letter Writer**, based on the previous implementation and the proposed improvements.

Brief: AI Cover Letter Writer (May Project)

Project Overview

This project aims to enhance the existing AI-powered cover letter generator by integrating a user-friendly **Flask-based front-end**. The improved tool will allow job seekers to **easily input their CV and job description** while ensuring **high-quality, naturally flowing cover letters** tailored to each application.

Previous Version (Limitations & Challenges)
The earlier implementation used a **Python script with Gemini AI models** to generate cover letters based on predefined prompts. While functional, it lacked:

- A **graphical user interface**, requiring manual script execution.
- **Customization options**, preventing users from refining the AI-generated content.
- **Interactivity**, meaning users couldn't receive real-time feedback or improve their cover letters before submission.

Proposed Improvements
The new version will introduce:
☑ **Web-Based UI** (Flask front-end) for seamless user interaction.
☑ **Enhanced Cover Letter Quality** with optimized AI prompts for better flow and readability.
☑ **Editable & Customizable Output** allowing users to refine their AI-generated cover letter before finalizing.

Implementation Plan

1. Flask Web Application

- Develop a simple and clean UI with **text input fields** for CV and job description.
- Ensure **mobile responsiveness** for accessibility on multiple devices.

2. AI Integration

- Utilize **Gemini 1.5 Flash-8B** for more natural and structured text generation.
- Optimize prompts to ensure cover letters **reflect professional tone and relevance**.

3. User Interactivity

- **Live preview** of AI-generated cover letters.
- **Real-time editing** before submission.

4. Additional Features

- **Copy-to-clipboard** for quick retrieval.
- **Download as PDF** for easy file sharing.
- **Potential email integration** for direct application submission.

Expected Outcome

- A **streamlined and automated process** for generating job application cover letters.
- **AI-driven yet human-like writing** that feels personal and professionally crafted.
- **Intuitive and efficient user experience** with minimal effort required.

This brief outlines the core objectives, challenges, and implementation roadmap for the **May AI Cover Letter Writer Project**. Would you like any refinements or additional details?

Conversation by Fifa Utuber with Copilot from Tuesday, Nov 19 2024 to Friday, May 2 2025